

CS 747: Programming Assignment 1

Margav Mukeshbhai Savsani

200050072

1. Task 1

(a) UCB

I implemented the algorithm as was mentioned in the class by maintaining empirical means of all arms, current time and how many times each arm has been pulled. Then using the condition of UCB pulled the appropriate arm. There were no major issues in implementing this but comparing the graphs of all 3 algorithms gave me a good idea on how much value good algorithms can add (although UCB is also much better as compared to epsilon greedy). The graph of Regret vs Horizon for UCB is logarithmic as was expected from theory



Figure 1: UCB

(b) KL-UCB

I implemented the algorithm as was mentioned in the class by maintaining empirical means of all arms, current time and how many times each arm has been pulled. I used binary search to find q value accurately upto the order of 10^{-4} using the condition of KL-UCB and used it to pull the appropriate arm. Initially I had taken q upto the accuracy of 10^{-6} but I did not find much of a difference in the results(on autograder). Also the graph of Regret vs Horizon is nearly logarithmic and the regrets are less than UCB as was expected.

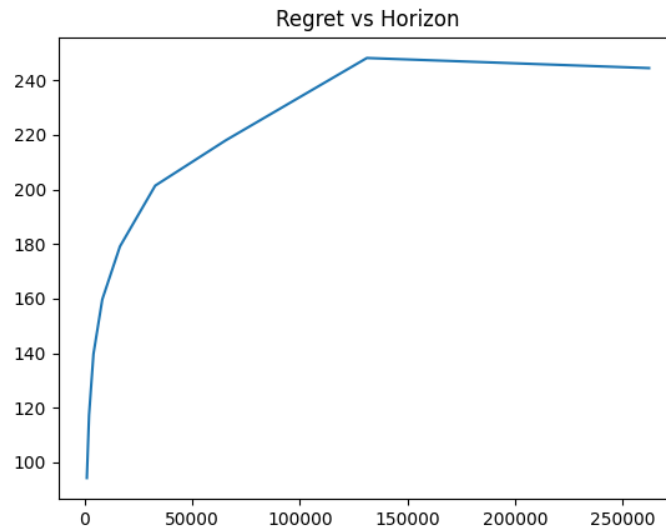


Figure 2: KL-UCB

(c) Thompson Sampling

I implemented the algorithm as was mentioned in the class by maintaining No. of successes and failures for each arm and then using beta distribution to get a value using which I pulled the appropriate arm. The implementation is rather simple but the results that it provided are extraordinary. I expected it to be better than others as it is backed by probability concepts but not to the level it came out to be. The regret is expected to increase with horizon but the graph seems to decrease after a point and the reason for this is that there can be some cases where the algorithm outperforms or underperforms in them which can lead to such disruptions in the long term pattern (graph not being perfectly logarithmic). Also the graph is plotted only for a few horizon values and then interpolated which lead to such a behaviour.



Figure 3: Thompson Sampling

2. Task 2

As the batch size can be any number and even 1 so taking the learnings from task 1 I modified Thompson sampling to fit for the task. So during each iteration when I was supposed to give batch_size no of predictions (arms to pull) I used Thompson sampling to decide for each of the prediction. Thus depending on how much confusion the algorithm has, if confusion is high it would explore the arms and if the confusion is low the predictions will get concentrated to specific arms. From the graph it is seen that regret increases as batch size increases which is justified as the number of times algorithm gets to learn from its mistakes is decreasing.



Figure 4: Task 2

3. Task 3

I implemented a modified version of epsilon greedy algorithm. I took into account 2 parameters to decide whether I should be exploring or exploiting at the given moment. The parameters are:

1) $v_1 = 1$ - The largest value of empirical mean

2) $v_2 = \frac{\text{No of pulls left}}{\text{Total No of pulls}}$

If any of the above parameters is low then I should be more into exploiting and if they are large I should explore. So I considered a expression $v_1(c_1v_2 + c_2v_2^2) < c_3$. If the expression is true I exploited else I explored a new arm. The values c_1, c_2, c_3 are hyperparameters which I tuned for the model. The graph is increasing but it performs much better then one pull per arm model.

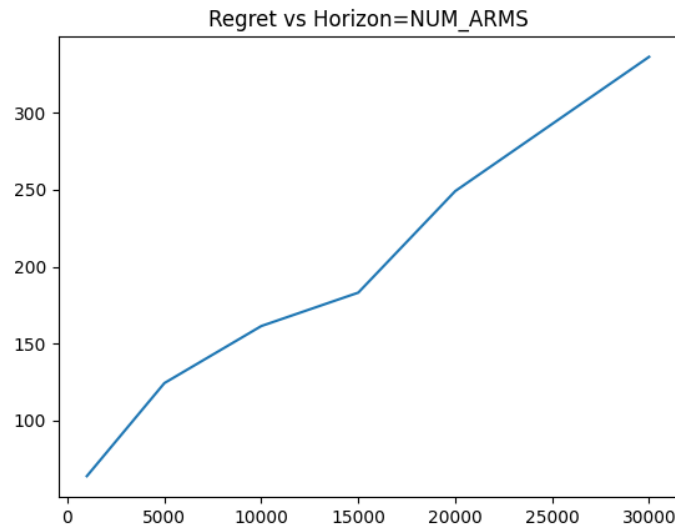


Figure 5: Task 3